

Java M2-A1 — Section A

Concept Application — Complete Scenario-Based Answers

This study PDF covers all six assessment scenarios with theory, practical examples, Java/JDBC examples, failure scenarios, and interview-ready conclusions.

Topics: Collections, Custom Exceptions, SQL Injection & PreparedStatement, JDBC Transactions, ResultSet Handling, and JDBC Batch Processing.

S1 — HashMap vs ArrayList for Active Orders

Scenario: Active orders are stored as `HashMap<String, Order>`, keyed by order ID. A junior developer suggests replacing it with `ArrayList<Order>` because it seems simpler.

Answer: `HashMap` is the better choice when the dominant operation is finding an order using its unique order ID. `ArrayList` is position-based and does not provide direct key lookup.

Operation	HashMap<String, Order>	ArrayList<Order>
Lookup by order ID	Average $O(1)$	$O(n)$ — scan every order
Insert new order	Average $O(1)$	$O(1)$ at end / $O(n)$ if inserted in middle
Remove by order ID	Average $O(1)$	$O(n)$ to find + $O(n)$ shifting; overall $O(n)$

Why `HashMap`? Suppose the system receives a request: "Show order ORD1042." With a `HashMap`, the program can directly use `orders.get("ORD1042")`. With an `ArrayList`, it must iterate through the list until it finds the matching ID.

```
Order order = orders.get("ORD1042");
```

Is switching a good idea? No, not for active-order lookup/removal by order ID. The switch would make the most important operations slower as the number of active orders grows.

Where `ArrayList` is better: A restaurant menu loaded in display order is a good example. If the application mainly needs to iterate through all menu items, preserve their order, and access them sequentially, `ArrayList<MenuItem>` is simple and appropriate.

Conclusion: Choose a collection based on the dominant access pattern, not simply on which API looks simpler.

S2 — Custom Exception Hierarchy for Payment Processing

Scenario: Payment processing has three failures: insufficient wallet balance, payment gateway timeout, and an item becoming unavailable during checkout.

What is a custom exception hierarchy? It is a set of application-specific exception classes arranged under a common parent. Instead of catching every failure as generic `Exception`, the code expresses what actually went wrong.

```
class PaymentException extends Exception {
    public PaymentException(String message) {
        super(message);
    }
}

class InsufficientBalanceException extends PaymentException {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}
```

```

class PaymentGatewayTimeoutException extends PaymentException {
    public PaymentGatewayTimeoutException(String message) {
        super(message);
    }
}

class ItemUnavailableException extends PaymentException {
    public ItemUnavailableException(String message) {
        super(message);
    }
}

```

Example meaning: `InsufficientBalanceException` means the customer's wallet cannot cover the order. `PaymentGatewayTimeoutException` means the external gateway did not respond in time. `ItemUnavailableException` means a menu item became unavailable before payment/order completion.

Why more maintainable? A generic catch such as `catch(Exception e)` hides the business meaning. A hierarchy lets the application react differently: show a balance message for insufficient funds, allow retry for a gateway timeout, and ask the customer to remove/replace an unavailable item.

```

try {
    processPayment();
} catch (InsufficientBalanceException e) {
    System.out.println("Please add money to your wallet.");
} catch (PaymentGatewayTimeoutException e) {
    System.out.println("Gateway timed out. Please retry.");
} catch (ItemUnavailableException e) {
    System.out.println("One item is no longer available.");
}

```

Conclusion: Custom exceptions improve readability, debugging, business-level error handling, and future extensibility.

S3 — SQL Injection and PreparedStatement

Scenario: Login and menu search queries are created by concatenating user input into SQL strings.

Vulnerable example:

```

String sql = "SELECT * FROM users WHERE username = '"
    + username + "' AND password = '" + password + "'";

Statement st = conn.createStatement();
ResultSet rs = st.executeQuery(sql);

```

Concrete attack: If an attacker enters a crafted value such as `' OR '1'='1` into a vulnerable text field, the resulting SQL can change the intended WHERE condition. In a poorly designed login query, this can make the condition logically true and bypass authentication. The exact impact depends on the original query and database syntax, but the root problem is that data has been merged into SQL code.

Why PreparedStatement fixes it: The SQL structure is defined separately from the values. User input is bound as data using placeholders, so characters in the input are not treated as SQL operators or clauses.

```

String sql =
    "SELECT * FROM users WHERE username = ? AND password = ?";

PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, username);
ps.setString(2, password);

ResultSet rs = ps.executeQuery();

```

Statement vs PreparedStatement: With Statement, the application builds the final SQL text first, often by concatenating user data, so the database receives a command whose syntax can be changed by malicious

input. With `PreparedStatement`, the SQL template contains parameters and the supplied values are bound separately, preventing them from becoming SQL syntax.

Menu search: The same rule applies: `SELECT ... WHERE name LIKE ?` with `ps.setString(1, "%" + keyword + "%")` is preferred over concatenating keyword into SQL.

Conclusion: `PreparedStatement` is both safer and cleaner for parameterized SQL and should be the standard approach for user-controlled values.

S4 — JDBC Transaction Management and Atomic Order Placement

Scenario: Three operations must succeed together: deduct wallet balance, insert order, and increment restaurant `pending_orders`.

Danger with auto-commit: Suppose the wallet deduction succeeds and is committed. Then the order `INSERT` fails because of a database error. The customer has paid, but no order exists. If the restaurant update fails afterward, the system can become even more inconsistent.

Required transaction pattern: Use one `Connection` and disable auto-commit before the three operations.

```
try (Connection conn = DriverManager.getConnection(URL, USER, PASSWORD)) {
    conn.setAutoCommit(false);

    try {
        // 1. Deduct wallet
        // 2. Insert order
        // 3. Increment pending_orders

        conn.commit();
    } catch (Exception e) {
        conn.rollback();
        throw e;
    }
}
```

Atomicity: Either all three changes become permanent, or none of them do. This is the key reason for using a transaction.

Failure scenario: If step 1 succeeds and step 2 fails, the catch block must call `rollback()`. Otherwise step 1 may remain committed and the wallet can be reduced without a corresponding order.

What if step 3 fails? The same rollback must undo both the wallet deduction and the order `INSERT`. All statements must use the same `Connection` because the transaction is associated with that connection.

Important: Always check business conditions such as insufficient balance. An `UPDATE` that matches zero rows may not throw `SQLException`, so the application should inspect the affected-row count and explicitly fail when the expected update did not occur.

S5 — ResultSet Column Access and SQL NULL

Scenario: A `ResultSet` contains `order_id`, `restaurant_name`, `total_amount`, `order_date`, and `status`. A developer reads values only by index.

Problem 1 — Fragility when `SELECT` changes: If a column is inserted, removed, or reordered in the `SELECT` list, positional indexes can start referring to different columns. Code can then read the wrong value or fail with a type mismatch.

Problem 2 — Poor readability/maintenance: `rs.getString(2)` does not communicate what column 2 means. Named access such as `rs.getString("restaurant_name")` is self-documenting and safer during maintenance.

```
int orderId = rs.getInt("order_id");
String restaurant = rs.getString("restaurant_name");
double amount = rs.getDouble("total_amount");
Timestamp date = rs.getTimestamp("order_date");
String status = rs.getString("status");
```

Preferred approach: Use column labels/names whenever practical. It makes the code easier to understand and reduces dependence on the exact SELECT column order.

SQL NULL: For object-returning getters such as `getString()` and `getTimestamp()`, SQL NULL is represented as Java null. For primitive getters such as `getInt()` and `getDouble()`, JDBC returns a default primitive value such as 0, so the application must check `rs.wasNull()` immediately after the getter when SQL NULL must be distinguished from a real zero.

```
double amount = rs.getDouble("total_amount");
if (rs.wasNull()) {
    System.out.println("Amount is NULL");
} else {
    System.out.println("Amount = " + amount);
}

String status = rs.getString("status");
if (status == null) {
    System.out.println("Status is NULL");
}
```

Conclusion: Named columns improve maintainability, and NULL-aware handling prevents incorrect business decisions caused by confusing SQL NULL with a numeric zero.

S6 — JDBC Batch Processing

Scenario: A report job inserts 500 daily summary rows. The current code executes one INSERT at a time inside a loop.

Without batching: The program calls `executeUpdate()` 500 times. Conceptually, this creates about 500 client-to-database execution cycles/round trips, depending on the JDBC driver and its optimizations.

With batching: The program adds 500 parameter sets to a batch and calls `executeBatch()`. This can greatly reduce communication overhead and lets the driver/database process multiple statements efficiently. The exact number of network round trips is driver- and configuration-dependent, so “one call” does not always literally mean exactly one network packet.

Why round trips matter: Each trip can involve network latency, request/response processing, statement handling, and synchronization. Repeating that overhead 500 times is usually much more expensive than sending work in larger batches.

```
conn.setAutoCommit(false);

PreparedStatement ps = conn.prepareStatement(
    "INSERT INTO report_logs(order_id, summary) VALUES (?, ?)"
);

for (...) {
    ps.setInt(1, orderId);
    ps.setString(2, summary);
    ps.addBatch();
}

int[] results = ps.executeBatch();

conn.commit();
```

Correct sequence: 1) optionally disable auto-commit, 2) prepare the INSERT, 3) set parameters, 4) call `addBatch()` for each row, 5) call `executeBatch()`, 6) inspect the returned update counts, and 7) call

commit() if the transaction is successful.

Partial success: A batch can encounter a failure after some commands have already succeeded. JDBC drivers can report update counts and may use special values such as Statement.EXECUTE_FAILED or Statement.SUCCESS_NO_INFO. Driver behavior around continuing after an error can vary.

```
int[] counts = ps.executeBatch();

for (int i = 0; i < counts.length; i++) {
    if (counts[i] == Statement.EXECUTE_FAILED) {
        System.out.println("Batch item " + i + " failed.");
    }
}
```

Handling strategy: For a report where all 500 rows must be inserted atomically, wrap the batch in a transaction and call rollback() when the batch fails. If partial completion is acceptable, log failed entries and retry them according to the application's requirements. The key is to inspect the returned update counts and handle BatchUpdateException rather than assuming every item succeeded.

```
try {
    int[] counts = ps.executeBatch();
    conn.commit();
} catch (BatchUpdateException e) {
    conn.rollback();
    System.out.println("Batch failed: " + e.getMessage());
    int[] counts = e.getUpdateCounts();
}
```

Quick Revision — 6 Answers

S1	HashMap	Fast key-based order lookup/removal; ArrayList for sequential menu/list display.
S2	Custom exceptions	Separate business failures into meaningful exception types.
S3	PreparedStatement	Keep SQL structure separate from user data; prevent SQL injection.
S4	Transactions	setAutoCommit(false) → operations → commit; failure → rollback.
S5	ResultSet	Prefer column names; handle SQL NULL with null checks / wasNull().
S6	Batch	addBatch() → executeBatch() → inspect counts → commit/rollback.

Exam tip: For scenario questions, answer in this order: identify the problem → explain the concept → show a small Java/JDBC example → describe the failure case → state the practical conclusion.